Software Requirements Specification

for

Smart Parking Managment System

Version 1.0 approved

Prepared by

Yajat Padmavat

Aditya Patil

Aryan Punalekar

TSEC

<25/07/26>

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents ii
Revision History ii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description 2
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 2
2.6 User Documentation 2
2.7 Assumptions and Dependencies 3
3. External Interface Requirements 3
3.1 User Interfaces 3
3.2 Hardware Interfaces 3
3.3 Software Interfaces 3
3.4 Communications Interfaces 3
4. System Features 4
4.1 System Feature 1 4
4.2 System Feature 2 (and so on) 4
5. Other Nonfunctional Requirements 4
5.1 Performance Requirements 4
5.2 Safety Requirements 5
5.3 Security Requirements 5
5.4 Software Quality Attributes 5
5.5 Business Rules 5
6. Other Requirements 5
Appendix A: Glossary 5
Appendix B: Analysis Models 5
Appendix C: To Be Determined List 6

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

The purpose of this software is to provide drivers, vehicle owners, and parking facility managers with a centralized platform for real-time tracking, reservation, and management of parking spaces . Traditionally, drivers spend considerable time cruising around congested urban areas searching for open parking spots, leading to traffic bottlenecks, fuel wastage, and increased emissions. To eliminate these inefficiencies, this software provides prior visibility into open parking slots and enables cashless pre-booking.

1.2 Document Conventions

- **Headings and Subheadings** are formatted in bold text for hierarchical clarity.
- **Crucial functional conditions** are emphasized in bold.
- Functional priorities follow a descending order from high to low priority across feature definitions.
- Standard software engineering terms and technical acronyms (e.g., API, JWT, MQTT) are referenced throughout and detailed in Appendix A.

1.3 Intended Audience and Reading Suggestions

This document is prepared for:

1. **Software Developers:** To guide the frontend, backend, and hardware-integration development process.
2. **Quality Assurance & Testing Team:** To formulate system test plans, acceptance criteria, and stress tests.
3. **Project Supervisors & Stakeholders:** To verify system functionality against academic and business criteria.

Reading Suggestion: Developers should start with **Section 2 (Overall Description)** followed by **Section 4 (System Features)**, while QA engineers should focus on **Section 3 (External Interfaces)** and **Section 5 (Nonfunctional Requirements)**.

1.4 Product Scope

The **Smart Parking Management System** is a cross-platform solution consisting of a responsive mobile/web application for drivers and a web portal for facility managers. Key features include:

- Pre-arrival real-time slot status inspection.
- Interactive parking spot selection and booking.
- Turn-by-turn navigation guidance to the exact parking spot.
- Automated contactless gate entry using QR codes or IoT sensor verification.
- Payment integration for instant digital tolling/checkout.

Future Enhancements: Automatic License Plate Recognition (ALPR) via AI vision, electric vehicle (EV) charger slot reservation, and predictive AI occupancy modeling

1.5 References

- **React Native / Web Docs:** https://reactnative.dev/
- **Node.js & Express Framework:** https://expressjs.com/
- **PostgreSQL Database Documentation:** https://www.postgresql.org/docs/
- **Google Maps Platform API:** https://developers.google.com/maps

2. Overall Description

2.1 Product Perspective

The Smart Parking Management System acts as a standalone platform connecting drivers directly with commercial and municipal parking infrastructure. By replacing manual paper ticket systems and guesswork with digital sensor streams and web-based slot allocation, the platform cuts entry/exit times, optimizes lot utilization, and eliminates middleman delays.

2.2 Product Functions

- **Live Slot Visibility:** View live counts of free, occupied, and reserved parking spaces prior to driving to the location.
- **Spot Selection & Pre-Booking:** Select precise parking slots (e.g., ground floor, near exit, EV charging slot).
- **Automated Check-in/out:** Mobile QR code generation for scanning at hardware barriers.
- **Integrated Payments:** Calculation of parking fees based on hourly rates with digital receipt issuance.
- **Administrative Operations:** Facility layout configuration, real-time analytics, dynamic pricing adjustments, and manual overrides.

2.3 User Classes and Characteristics

1. **Drivers / Vehicle Owners (Primary Users):** Access live counts, reserve slots, navigate, pay, and track booking history.
2. **Parking Attendants / Facility Managers:** Oversee physical lot spaces, handle manual entry/exit exceptions, and inspect physical sensors.
3. **System Administrators:** Manage user accounts, approve new parking facility additions, set platform service fees, and view financial reports.

2.4 Operating Environment

- **Client Devices:** Mobile devices (iOS 13+, Android 8.0+) and desktop browsers.
- **Web Browsers:** Google Chrome 90+, Mozilla Firefox 85+, Apple Safari 14+, Microsoft Edge.
- **Backend Platform:** Node.js runtime environment hosted on cloud infrastructure (e.g., AWS / Linux Ubuntu 22.04 LTS Server).
- **Database & Memory Cache:** PostgreSQL 15+ for relational records; Redis 7+ for real-time slot state management.
- **Network Minimum:** 3G / 4G / 5G / Wi-Fi connection with at least 1 Mbps throughput.

2.5 Design and Implementation Constraints

- **CO-1:** Project development schedule is capped at 3 months.
- **CO-2:** Frontend must be built using React / React Native for seamless cross-platform performance.
- **CO-3:** Backend REST APIs and WebSockets must be built on Node.js / Express.
- **CO-4:** System interfaces are provided in English.
- **CO-5:** Hardware interaction with gate barriers requires MQTT standard protocol compliance.

2.6 User Documentation

The system will supply user onboarding flows within the mobile application. Context-sensitive help dialogues will assist during reservation, payment, and QR generation. Administrators will receive an inline operational manual covering lot setup and pricing rule adjustments.

2.7 Assumptions and Dependencies

- **AS-1:** Users possess a GPS-enabled smartphone with an active internet connection.
- **AS-2:** Parking lots are equipped with internet-connected entry/exit scanners or ultrasonic/optical slot sensors.
- **DE-1:** Map rendering depends on Google Maps JavaScript and Mobile SDK APIs.
- **DE-2:** Payment handling depends on payment gateway integration (Razorpay / Stripe).

3. External Interface Requirements

3.1 User Interfaces

- **UI-1 (Landing & Map View):** Interactive visual map displaying color-coded pins for parking locations (Green = High availability, Yellow = Limited, Red = Full).
- **UI-2 (Slot Layout Screen):** Visual grid representing individual physical spaces (Available, Reserved, Occupied).
- **UI-3 (Navigation Bar):** Persistent top/bottom navigation bar for quick access to Map, Bookings, Wallet, and Profile.
- **UI-4 (Alerts & Notifications):** Pop-up alerts indicating expiring reservations, successful payments, or pathing warnings.

3.2 Hardware Interfaces

- **HI-1 (Gate Barrier Controller):** Microcontroller units (ESP32/Raspberry Pi) controlling physical barrier arms receiving open/close signals over MQTT.
- **HI-2 (QR Scanners):** Optical scanners at entry/exit points connected via HTTP API endpoints to validate user passes.
- **HI-3**: Ultrasonic/IR proximity sensors installed per parking slot, communicating occupancy data via MQTT or LoRaWAN
- **HI-4**: Outdoor LED display panels showing available slot counts, connected via RS-485 or Ethernet

3.3 Software Interfaces

- **Operating Systems:** Android, iOS, Windows, macOS, Linux.
- **Database Systems:** PostgreSQL database server connected via SSL.
- **External APIs:** Google Maps API (Geocoding & Directions), Payment Gateway APIs.

3.4 Communications Interfaces

- **HTTPS/TLS 1.3:** Secure encryption protocol for all user transactions and REST endpoint interactions.
- **WebSockets (WSS):** Low-latency duplex connection for pushing real-time slot state changes to client screens.
- **MQTT:** Lightweight messaging protocol for sensor data exchange between backend servers and facility gate hardware.

4. System Features

4.1 Authentication and Authorization

4.1.1 Description and Priority

High Priority. Secures user accounts, protects payment information, and enforces role-based permissions (Driver, Manager, Admin).

4.1.2 Response Sequences

1. User enters phone number/email and password.
2. System validates input, verifies hashed credentials against PostgreSQL, and issues a JSON Web Token (JWT).
3. User is directed to the primary map interface.

4.1.3 Functional Requirements

- **REQ-1: Passwords must be hashed using bcrypt prior to storage.**
- **REQ-2: Passwords must be at least 8 characters long, containing 1 numeric character and 1 special character.**

4.2 Real-time Parking Availability Search

4.2.1 Description and Priority

High Priority (Core Feature). Allows drivers to inspect exact open slot counts across parking facilities prior to arrival.

4.2.2 Response Sequences

1. User enters a location or enables current GPS location.
2. System fetches nearby facilities and streams live availability data via WebSockets.
3. User taps a facility to inspect floor plans and open slots.

4.2.3 Functional Requirements

- **REQ-1:** Live slot updates must synchronize with client views within 2 seconds of status change.

4.3 Advance Slot Reservation

4.3.1 Description and Priority

High Priority. Enables users to reserve a chosen slot for a specified timeframe.

4.3.2 Response Sequences

1. User selects a free slot from the floor grid, specifies arrival time and duration, and confirms.
2. System places a temporary 15-minute lock on the slot.
3. Upon payment confirmation, the slot status changes to "Reserved".

4.3.3 Functional Requirements

- **REQ-1:** System must prevent double-booking by locking selected slots atomically in Redis during checkout.

4.4 Navigation & Directions Feature

4.4.1 Description and Priority

Medium Priority. Guides the user from their current GPS location directly to the selected parking facility entrance.

4.4.2 Response Sequences

1. User taps "Navigate" on an active booking.
2. System invokes Google Maps API route overlay within the app.

4.4.3 Functional Requirements

- **REQ-1:** Application must launch external navigation apps (Google Maps, Apple Maps) as a fallback option.

4.5 QR Code Entry & Exit Gate Processing

4.5.1 Description and Priority

High Priority. Facilitates contactless access to parking facilities.

4.5.2 Response Sequences

1. User presents active booking QR code to the scanner at the entry gate.
2. Hardware posts QR payload to backend API for validation.
3. System verifies booking, records check-in timestamp, and triggers gate signal to open.

4.5.3 Functional Requirements

- **REQ-1:** QR codes must use cryptographic timestamps to prevent reuse or static screenshot spoofing.

4.6 Automated Payment Processing

4.6.1 Description and Priority

High Priority. Handles digital payments for reservations and overstay fees.

4.6.2 Response Sequences

1. System calculates total charges based on duration upon exit scanning.
2. Payment modal prompts user to complete transaction via UPI, card, or wallet.
3. Upon success, digital receipt is issued and gate opens.

4.6.3 Functional Requirements

- **REQ-1:** Payment gateway webhooks must update transaction states asynchronously.

4.7 Facility Management & Dynamic Pricing (Admin)

4.7.1 Description and Priority

Medium Priority. Allows managers to adjust pricing rates and block out slots for maintenance.

4.7.2 Response Sequences

1. Manager logs into Admin Portal, selects a facility, and adjusts base hourly rates.
2. Changes save instantly and reflect on driver search views.

4.7.3 Functional Requirements

- **REQ-1:** Rate adjustments cannot affect pre-existing confirmed reservations.

4.8 Booking History & Receipt Generation

4.8.1 Description and Priority

Low Priority. Provides historical logs of past parking sessions with downloadable invoices.

4.8.2 Response Sequences

1. User opens Profile > Booking History.
2. System renders historical list with date, cost, location, and PDF download button.

4.8.3 Functional Requirements

- **REQ-1:** Receipts must be rendered dynamically in PDF format upon request.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- **PR-1:** System must support at least 1,000 concurrent WebSocket connections per node without degrading response times beyond 2 seconds.
- **PR-2:** Database queries for slot availability must execute within 100ms under normal traffic conditions.

5.2 Safety Requirements

- **SR-1:** In the event of network or server failure, physical gate barriers must default to manual operation mode to prevent vehicle entrapment.
- **SR-2:** Hardware API tokens and database connection strings must be stored securely using environment variables and secret managers.

5.3 Security Requirements

- **SEC-1:** All database records storing user passwords must be hashed using bcrypt with a salt factor of 10
- **SEC-2:** Transport Layer Security (TLS 1.3) must be enforced across all communications.
- **SEC-3:** Role-based access control (RBAC) must restrict administrative endpoints to authenticated admin tokens only.

5.4 Software Quality Attributes

- **5.4.1 Usability:** Interface layout must adhere to standard mobile design guidelines, requiring at most 3 taps to reach reservation screens.
- **5.4.2 Availability:** System architecture should aim for 99.9% uptime using cloud container auto-scaling.
- **5.4.3 Maintainability:** Modular code structure with thorough API documentation (OpenAPI/Swagger) to simplify future code additions.
- **5.4.4 Testability:** Minimum 80% code coverage across unit and integration testing pipelines.

5.5 Business Rules

- **BR-1:** Reservations hold a 15-minute grace period past the booked start time. If the vehicle fails to enter, the reservation is canceled and a nominal fee is charged.
- **BR-2:** Vehicles overstaying their booked window are billed at an escalated penalty rate per additional hour.

6. Other Requirements

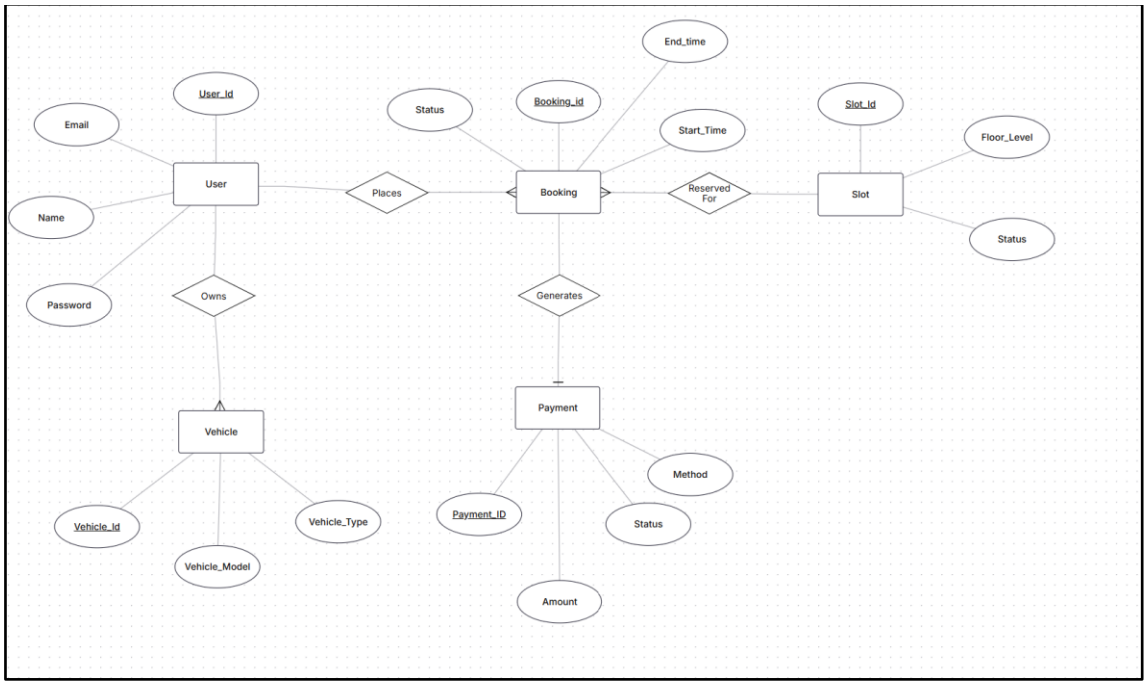
- **Localization:**System timestamps and currency formats must automatically adapt to regional defaults (e.g., IST / INR) .

Appendix A: Glossary

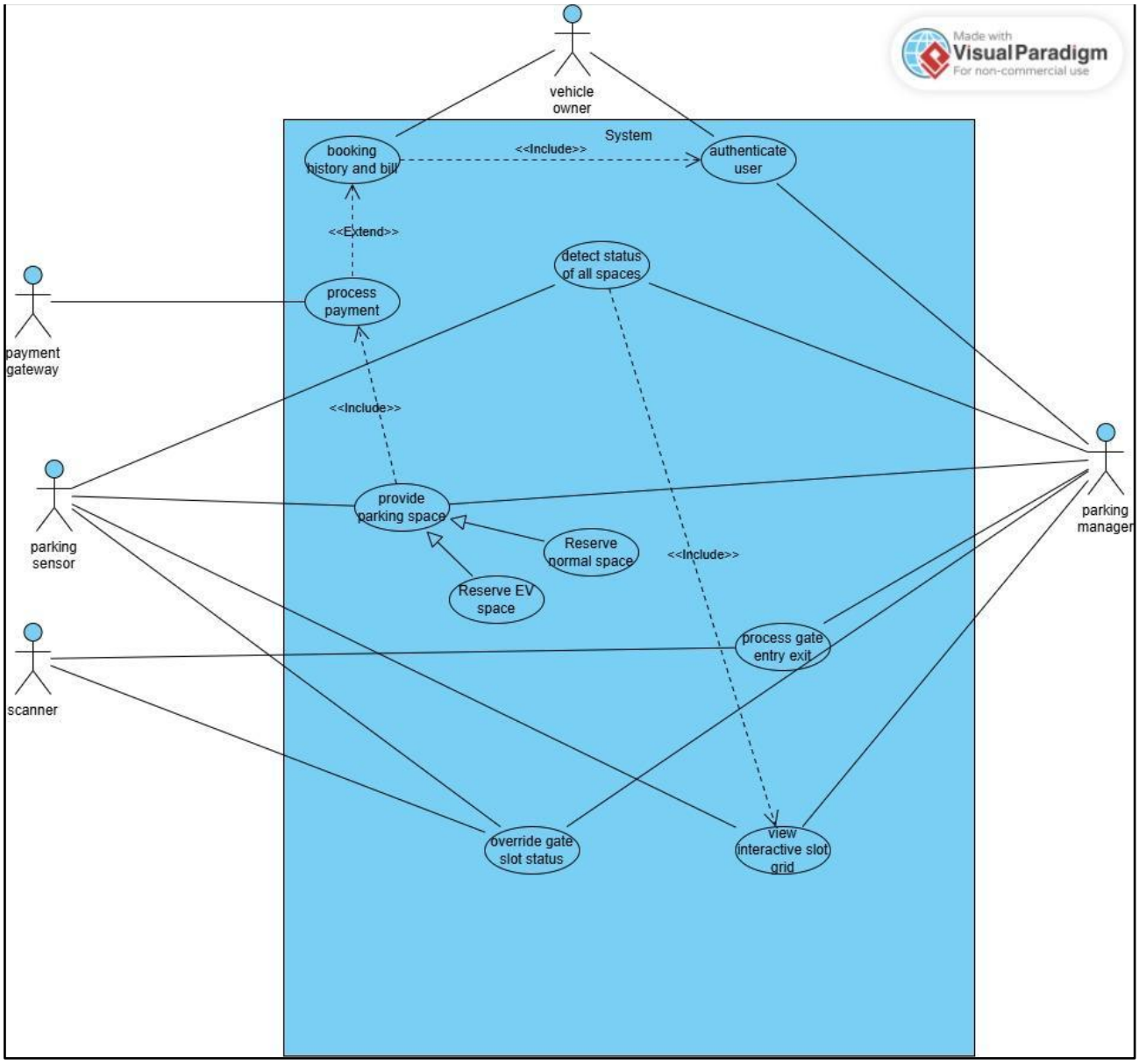
- **API:** Application Programming Interface.
- **GUI:** Graphical User Interface.
- **HTTPS:** Hypertext Transfer Protocol Secure.
- **JWT:** JSON Web Token (Used for secure authentication)
- **MQTT:** Message Queuing Telemetry Transport (Lightweight messaging protocol for IoT sensors).
- **WebSocket:** Computer communications protocol providing full-duplex communication channels over a single TCP connection.

Appendix B: Analysis Models

a) ER diagram



b) Use Case Diagram



Appendix C: To Be Determined List

<Collect a numbered list of the TBD (to be determined) references that remain in the SRS so they can be tracked to closure.>